

Magnus v0

Snapshot do modelo de visão computacional do Magnus em sua primeira geração funcional, antes do fine-tuning com dataset expandido. Documento gerado para registrar a baseline e permitir comparação objetiva com versões futuras.

1. Resumo executivo

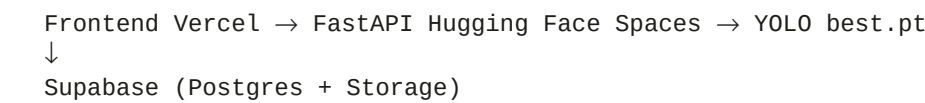
O Magnus é um sistema de inspeção visual automatizada da pintura de chassis estilo BAJA SAE. Capta imagens via webcam ou upload, executa detecção de defeitos em servidor de IA hospedado em nuvem e devolve bounding boxes + classes + confiança em tempo quase real. Esta é a versão **v0** — primeira iteração com modelo fine-tunado em dataset público real, sucessora da fase *demo* que rodava YOLO genérico (COCO) com remapeamento simbólico de classes.

Status atual

Componente	Status
Backend (FastAPI + YOLO)	Em produção — Hugging Face Spaces
Frontend (Next.js + Magnus UI)	Em produção — Vercel
Banco de dados	Supabase Postgres + Storage (1.999 imagens)
Modelo	YOLOv8n fine-tuned, 4 classes, mAP50 = 0.989
Origem do checkpoint	hf://Guguinhaxd/baja-paint-models/best.pt
Latência observada	~60 ms inferência CPU · ~350 ms round-trip
Custo mensal de infra	R\$ 0,00 (free tier em todos os serviços)

2. Arquitetura

Pipeline cloud serverless em três serviços conectados, com modelo versionado em hub externo:



Convenções importantes

Esquema de pesos hf:// — implementação custom no backend (backend/app/inference.py) aceita BAJA_MODEL_WEIGHTS=hf://owner/repo/file.pt. No boot do Space, baixa o checkpoint via huggingface_hub, cacheia em BAJA_MODEL_CACHE_DIR e carrega no Ultralytics. Permite hot-swap de modelo trocando uma variável de ambiente.

Separação de credenciais — service_role do Supabase só no backend (escrita autenticada via JWT admin); anon key no frontend (leitura via RLS pública).

Modo demo automático — se o checkpoint carregado for um yolov8/yolo11/yolov5 stock, a resposta marca demo_mode: true e aplica DEMO_CLASS_MAP; com checkpoint fine-tunado, demo_mode é false e os rótulos vêm direto do modelo.

3. Modelo v0 — detalhes técnicos

Atributo	Valor
Arquitetura	YOLOv8n (Ultralytics)
Parâmetros (treináveis)	3.011.628
Parâmetros (após fusão para inferência)	3.006.428
Camadas	130 (73 após fusão)
GFLOPs por inferência	8,2
Tamanho em disco	6,25 MB (best.pt)
Classes (4)	casca_de_laranja · escorrimento · bolha · water_spotting
Origem do pré-treino	yolov8n.pt (COCO)
Dataset de fine-tuning	baopersonal/paint-defect-detection-lx0xk (Roboflow)
Tamanho do dataset	2.000 imagens (1.600 train / 399 val)
Hardware de treino	Colab T4 GPU (free tier)
Epochs	80 (com early stopping)
Image size	640
Batch size	16
Tempo total de treino	~42 min

4. Métricas v0 (conjunto de validação)

Métricas globais

Métrica	Valor
mAP@50	0,989
mAP@50–95	0,864
Precision (média)	0,970
Recall (média)	0,966

Por classe (mAP@50)

Classe	mAP@50	mAP@50–95	Instâncias val
bolha	0,985	0,828	116
casca_de_laranja	0,995	0,919	93
escorrimento	0,992	0,850	84
water_spotting	0,987	0,857	106

Leitura honesta. O mAP elevado reflete um conjunto de validação restrito ao mesmo domínio do treino (closeups industriais com iluminação controlada, do Roboflow Universe). Em fotos de chassi inteiro com câmera de celular o desempenho cai sensivelmente — caracterizando lacuna de generalização (domain gap). É essa lacuna que o ciclo de fine-tuning v1 pretende reduzir.

5. Stack tecnológica

Camada	Tecnologia	Hospedagem
Frontend	Next.js 14 + TypeScript + Tailwind	Vercel
Identidade visual	Inter font · paleta verde sóbria · logo SVG de coroa de louros	
Backend	FastAPI + Uvicorn + Pydantic v2	Hugging Face Spaces (Docker)
Inferência	Ultralytics YOLO (PyTorch)	Mesmo container
Banco	Supabase Postgres (RLS público read-only)	Supabase
Storage	Supabase Storage (buckets privados)	Supabase
Modelos	Hugging Face Hub (repos públicos)	Hugging Face
Treino	Colab notebook + Ultralytics	Google Colab T4
Versionamento	Git em GitHub	github.com/gustavo-dev04/baja
Autenticação	JWT HS256 admin password-based	Backend
CORS	lista exata + regex (cobre previews Vercel)	Backend

6. Dataset v0

Atributo	Valor
Fonte única	Roboflow Universe — baopersonal/paint-defect-detection-lx0xk
Licença	CC BY 4.0
Imagens totais	2.000
Distribuição por classe	Balanceada — 500 imagens por classe
Splits	Train 1.600 · Validation 399 · Test 1
Resolução	640×640 (preprocess Roboflow)
Anotação	Bounding boxes YOLO (xywh normalizado)
Domínio	Closeups industriais de superfície pintada
Persistência no projeto	Importado para Supabase via training/import_roboflow.py
Dataset id (Supabase)	paint-defects-pretrain

7. API pública (contratos preservados em v1)

Método	Endpoint	Descrição
GET	/health	Status + nome do modelo + demo_mode
POST	/inspect	Recebe multipart com 'file', devolve InspectionResult
POST	/api/v1/auth/admin-token	Troca senha por JWT (24h)
GET	/api/v1/datasets	Lista pública de datasets
POST	/api/v1/datasets	Cria dataset (admin)

POST	/api/v1/datasets/{id}/images	Upload bulk (até 20, admin)
GET	/api/v1/images/{id}/annotations	Lista anotações
POST	/api/v1/images/{id}/annotations	Replace-all (admin)

Formato da resposta /inspect (resumido)

```
{
  "detections": [
    {
      "label": "casca_de_laranja",
      "raw_label": "casca_de_laranja",
      "confidence": 0.95,
      "bbox": [x1,y1,x2,y2]
    }
  ],
  "image_size": [w,h],
  "inference_ms": 66.0,
  "model": "hf://Guguinhaxd/baja-paint-models/best.pt",
  "demo_mode": false
}
```

8. Limitações conhecidas (gaps que v1 atacará)

Limitação	Causa	Plano para v1
Apenas 4 classes detectadas	Dataset v0 só tinha 4 classes nomeadas	Expandir para 8 classes via união curada de 6 datasets
Generalização ruim para fotos zóneas	Dataset v0 é só closeup industrial	Adicionar fotos próprias do chassi BAJA na Fase 3
Sem persistência de inspeções	Handler /inspect não grava no banco	Adicionar persistência após v1 estar estável
replace_annotations não transa	Supabase SDK sem suporte a transações	Migrar para Postgres function (RPC) se virar problema
@app.on_event deprecated	FastAPI moveu para lifespan	Migrar para context manager lifespan
3 classes do schema sem treino	falha_cobertura, risco, oxidacao sem dataset	Adicionar sobre risco e oxidacao via car-paint-damage; falha_cobertura

9. O que muda em v1 (fine-tuning planejado)

Aspecto	v0 (este snapshot)	v1 (próximo)
Arquitetura	YOLOv8n	YOLO11s
Parâmetros	3,0 M	9,0 M
Classes	4	8
Fontes de dataset	1 (1x0xk)	6 curadas com dedup cross-source
Receita de treino	default Ultralytics	cosine LR + MixUp + CopyPaste + warmup + close_mosaic + overs
Avaliação	mAP simples no val	TTA + matriz de confusão + per-class no test split
Tracking	logs stdout	Weights & Biases opcional
Pipeline location	training/	training/v2/
Modelo no Hub	Guguinhaxd/baja-paint-models	Guguinhaxd/magnus-v2
mAP@50 esperado	0,989 (dataset de validação restrito)	0,80–0,90 (mais classes, mais difícil — número honesto)

Insight metodológico

Auditoria de 13 datasets públicos identificou que **~68% do volume aparente é ruído de domínio** (carros capotados, amassados, rodas) ou augmentation embutido (3× por imagem-fonte). O pool útil curado é de ~3.000–5.500 imagens, dependendo do overlap entre versões do mesmo autor. A contribuição científica defensável de v1 não é *volume* mas *curadoria*: demonstrar que dataset pequeno e bem rotulado supera dataset grande e ruidoso.

10. Referências de código (commits-âncora)

Branch ativa: `claude/paint-inspection-ai-FRk8g`
Repositório: `github.com/gustavo-dev04/baja`

Componente	Arquivos principais
Inferência (hf:// scheme)	<code>backend/app/inference.py</code> · <code>backend/app/core/config.py</code>
Routers da API	<code>backend/app/routers/{inspect,auth,datasets,images,annotations}.py</code>
Auth JWT	<code>backend/app/core/auth.py</code>
UI Magnus	<code>frontend/app/page.tsx</code> · <code>components/Logo.tsx</code> · <code>components/ResultsPanel.tsx</code>
Pipeline v0 (treinou este modelo)	<code>training/import_roboflow.py</code> · <code>training/train_yolo.py</code>
Pipeline v1 (próximo treino)	<code>training/v2/{config,build_dataset,train,evaluate}.py</code>
Schema do banco	<code>supabase/migrations/0001_init.sql</code>
Deploy	<code>DEPLOY.md</code> · <code>backend/Dockerfile</code>

Gerado em 2026-06-09 22:14 · Documento auto-suficiente, destinado a outro projeto Claude Code que registrará a progressão entre versões do Magnus.